

PAYROLL MANAGEMENT SYSTEM

Team Build Plan, Architecture & Learning Roadmap

Kenya-first | Node.js + TypeScript + Express | PostgreSQL + Prisma | Next.js | BullMQ/Redis | GCP Cloud Run
Version 1.1 | September 2026

Purpose. A sequential implementation roadmap for a payroll platform covering employee setup, compensation, calculations, approval, payslips, reports, automation, storage, email and deployment.

Planning estimate: 175 working days, sequentially. Deadlines are task estimates, not calendar dates. If work slips, move the schedule rather than skipping prerequisites or bug-fix windows.

1. Chosen Stack & System Architecture

Build a **modular monolith** in a pnpm monorepo. Keep the API, web app and background worker as separate deployable containers, not separate microservices.

Area	Reference / purpose
Backend	Node.js + TypeScript + Express
Database / ORM	PostgreSQL / Prisma
Validation	Zod
Background jobs	BullMQ + Redis
PDF generation	Puppeteer or @react-pdf/renderer — choose one for MVP
Email	Nodemailer + SendGrid
Scheduling	BullMQ repeatable jobs / Job Schedulers; replaces system cron
Auth	JWT + bcrypt; follow Njirani's architectural pattern, never reuse secrets
Frontend	Next.js + Tailwind CSS + shadcn/ui
File storage	GCP Cloud Storage for payslip PDFs
Deployment	Docker + GCP Cloud Run
CI/CD	GitHub Actions

System flow

Next.js web → Express API → Prisma → PostgreSQL
Express API → BullMQ → Redis → worker
Worker → PDF renderer → GCP Cloud Storage
Worker → Nodemailer → SendGrid

BullMQ note: Current BullMQ docs use the term Job Schedulers for recurring scheduling. Use the supported API for the version pinned in the project rather than copying legacy examples. **Cloud Run note:** validate lifecycle/scaling settings for a long-running queue worker before production.

2. Monorepo Structure

```
payroll-platform/  
■■■ apps/api/ # Express + TypeScript API  
■■■ apps/worker/ # BullMQ workers  
■■■ apps/web/ # Next.js frontend  
■■■ packages/database/ # Prisma schema/client/migrations  
■■■ packages/contracts/ # shared API/domain contracts  
■■■ packages/validation/ # shared Zod schemas  
■■■ packages/ui/ # shadcn/ui components  
■■■ packages/email/ # Nodemailer + SendGrid  
■■■ packages/storage/ # GCP Cloud Storage  
■■■ packages/config/ # shared tooling configuration  
■■■ packages/test-utils/ # fixtures and test helpers  
■■■ docs/ # requirements, ADRs, rules, API, runbooks  
■■■ infra/ # Docker and CI configuration  
■■■ pnpm-workspace.yaml / package.json / README.md
```

Express layering

Routes → middleware → controllers → services/use-cases → Prisma data access. Controllers handle HTTP translation; services contain business logic. Payroll calculations do not belong in controllers or React components.

3. Domain Rules

Organization → Employee → Employment Contract → Compensation → Payroll Inputs → Payroll Run → Calculation → Approval → Finalization → Payslip/Reports → Storage/Email

- **PostgreSQL is the system of record.** Use relational constraints and transactions for payroll-critical operations.
- **Finalized payroll is immutable.** Correct mistakes through explicit adjustments/reversals, not silent edits.
- **Statutory rules are versioned/effective-dated.** Store official source, effective date and tests.
- **Use decimal-safe money handling** with explicit precision and rounding.
- **Audit sensitive mutations** and enforce tenant authorization server-side.
- **Validate input with Zod.** Queue jobs must be idempotent so retries cannot duplicate payroll, PDFs or emails.
- **Njirani is a pattern reference only:** never reuse its secrets, keys or environment values.

Suggested state flow: DRAFT → CALCULATING → CALCULATED → PENDING_APPROVAL → APPROVED → FINALIZED → PAID. Model and audit cancellation/reversal paths explicitly.

4. Sequential Build Plan & Deadlines

Complete each phase in order. Every task deadline below is a working-day estimate. The stabilization tasks are reserved time, not optional padding.

Phase 0 — Product Discovery & Architecture

Allocation: 10 days

Task	Deadline
Requirements, glossary, MVP scope	2 days
Roles/RBAC matrix	1 day
Workflows/state diagrams	2 days
Domain model + ERD	2 days
Architecture Decision Records	1 day
Monorepo bootstrap	2 days

Phase 1 — Platform Foundation

Allocation: 10 days

Task	Deadline
Environment/configuration	1 day
PostgreSQL + Prisma setup/migrations	2 days
Express middleware/error conventions	1 day
JWT + bcrypt authentication	2 days
RBAC middleware	2 days
Audit-log foundation	2 days

Phase 2 — Organization & Employees

Allocation: 15 days

Task	Deadline
Organization schema/API	2 days
Department schema/API	1 day
Employee Prisma model	2 days
Employee Zod schemas	3 days
Employee service/controller/routes	2 days
Employee frontend	2 days
Tests	2 days
Bug-fix window	1 day

Phase 3 — Employment & Compensation

Allocation: 15 days

Task	Deadline
Employment contracts	2 days
Salary structures	2 days
Earnings/allowances	2 days
Deductions	2 days
Effective-dated compensation rules	2 days
Compensation UI	2 days
Integration tests	2 days
Bug-fix window	1 day

Phase 4 — Payroll Inputs

Allocation: 15 days

Task	Deadline
Attendance/input model	2 days
Overtime/extra earnings	2 days
Bonuses/commissions	2 days
Deductions/adjustments	2 days
Taxable/non-taxable classification	2 days
Input API + Zod validation	2 days
Input UI	2 days
Bug-fix window	1 day

Phase 5 — Calculation Engine

Allocation: 20 days

Task	Deadline
Engine interfaces/rule pipeline	2 days
Gross earnings	2 days
Deduction pipeline	2 days
PAYE	3 days
NSSF	2 days
SHIF	2 days
Housing Levy	2 days
Reliefs/exemptions/effective-date resolver	2 days
Integration + golden tests	2 days
Bug-fix window	1 day

Phase 6 — Payroll Run, Approval & Finalization

Allocation: 15 days

Task	Deadline
Payroll run/period locking	2 days
Batch employee selection	1 day
Calculation orchestration	2 days
Validation/exception reporting	2 days
Approval workflow + RBAC	2 days
Immutable finalization snapshot	2 days
Payroll run UI	2 days
Bug-fix window	2 days

Phase 7 — Payslips & Reporting

Allocation: 15 days

Task	Deadline
Payslip model/data contract	1 day
PDF template/generation	2 days
GCP Cloud Storage integration	2 days
Payroll register	2 days
PAYE report	2 days
NSSF report	2 days
SHIF report	2 days
P9 report	1 day
Bug-fix window	1 day

Phase 8 — Jobs, Scheduling & Email

Allocation: 10 days

Task	Deadline
Redis + BullMQ setup	2 days
Payslip/report worker	2 days
BullMQ recurring schedules; no system cron	2 days
Nodemailer + SendGrid	2 days
Retry/idempotency	1 day
Bug-fix window	1 day

Phase 9 — Frontend & Self-Service

Allocation: 15 days

Task	Deadline
Next.js shell/dashboard	2 days
Tailwind theme/layout	1 day
shadcn/ui setup	1 day
Employee profile	2 days
Payroll review	3 days
Reports center	2 days
Payslip history/download	2 days
Responsive/accessibility pass	1 day
UI bug-fix window	1 day

Phase 10 — Security & Hardening

Allocation: 15 days

Task	Deadline
Security headers/rate limiting/input hardening	2 days
Tenant-isolation tests	2 days
Audit review	1 day
Structured API/worker logging	2 days
Health/readiness checks	1 day
Cloud Run service config	2 days
Storage IAM/access review	1 day
Database backup/restore procedure	2 days
Dependency/container scan	1 day
Bug-fix window	1 day

Phase 11 — CI/CD & Release Infrastructure

Allocation: 10 days

Task	Deadline
GitHub Actions PR CI	2 days
Database/integration test pipeline	2 days
Docker builds	2 days
Cloud Run staging	1 day
Cloud Run production	1 day
Rollback procedure	1 day
Release runbook	1 day

Phase 12 — Final QA & MVP Release

Allocation: 10 days

Task	Deadline
End-to-end suite	3 days
Regression pass	2 days
Performance/load verification	1 day
Security regression	1 day
Documentation/handover	1 day
Release-candidate bug-fix window	2 days

5. Learning & Official References

Use official documentation first and match each reference to the dependency version pinned in the repository.

Area	Reference / purpose
Node.js	https://nodejs.org/docs/latest/api/
TypeScript	https://www.typescriptlang.org/docs/
Express	https://expressjs.com/
PostgreSQL	https://www.postgresql.org/docs/
Prisma	https://www.prisma.io/docs/orm
Zod	https://zod.dev/
BullMQ	https://docs.bullmq.io/
Job Schedulers	https://docs.bullmq.io/guide/job-schedulers/
Redis	https://redis.io/docs/latest/
Puppeteer	https://pptr.dev/
@react-pdf/renderer	https://react-pdf.org/
Nodemailer	https://nodemailer.com/
SendGrid	https://www.twilio.com/docs/sendgrid
Next.js	https://nextjs.org/docs
Tailwind	https://tailwindcss.com/docs
shadcn/ui	https://ui.shadcn.com/docs
Docker	https://docs.docker.com/
GitHub Actions	https://docs.github.com/en/actions
Cloud Run	https://cloud.google.com/run/docs
Cloud Storage	https://cloud.google.com/storage/docs
OWASP ASVS	https://owasp.org/www-project-application-security-verification-standard/
OWASP Password Storage	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
JWT RFC 7519	https://www.rfc-editor.org/rfc/rfc7519

Kenya statutory references

Verify current statutory rules before every production release. Record source, effective date and test cases; old tutorials and old rates are not authoritative.

Area	Reference / purpose
KRA — PAYE	https://www.kra.go.ke/
NSSF	https://www.nssf.or.ke/
Social Health Authority — SHIF/SHA	https://sha.go.ke/
Kenya Government	https://www.kenya.go.ke/

6. Testing, Security & CI/CD

Area	Reference / purpose
Unit tests	Calculations, validators, rounding, effective dates and edge cases.
Integration tests	Express + service + Prisma/PostgreSQL interactions.
Queue tests	Retries, idempotency and worker failure recovery.
PDF tests	Correct employee/period/fields and successful storage upload.
Email tests	Template rendering and delivery failure handling.
Authorization tests	Role permissions and cross-organization access attempts.
E2E tests	Employee → payroll run → approval → finalization → payslip → report.
Regression	Turn important production bugs into regression tests.

Security rules

- Hash passwords with bcrypt; never store plaintext passwords.
- Keep JWT secrets in managed secrets, never Git.
- Enforce API authorization and tenant scoping server-side.
- Do not put sensitive payroll data in logs.
- Restrict Cloud Storage objects; use authenticated downloads or controlled signed URLs.
- Use least-privilege IAM and audit sensitive actions.
- Run dependency/container checks in CI/CD.

CI/CD flow

Pull request: install → lint → typecheck → unit tests → integration tests → build → Docker validation
Staging: build images → deploy web/API/worker → reviewed migration procedure → smoke tests
Production: approved release → deploy Cloud Run → health checks → smoke tests → monitor/rollback

7. Team Rules, Milestones & Definition of Done

- One focused feature branch and small PR per feature.
- Every feature has acceptance criteria, tests and documentation.
- Schema changes go through reviewed Prisma migrations; no direct production edits.
- Record official source/effective date for statutory changes.
- No premature microservices or business logic in controllers/UI.
- Document architecture changes as ADRs; stabilize prerequisites before dependent work.

Milestones

Area	Reference / purpose
M1	Foundation: monorepo, Express, Prisma/PostgreSQL, auth, RBAC, audit.
M2	Organization, employee, contract and compensation domains.
M3	Validated payroll inputs.
M4	Reproducible calculation engine with golden tests.
M5	Approval and immutable finalization.
M6	Payslips/reports generated, stored and retrieved.
M7	BullMQ scheduling, workers and transactional email.
M8	Admin and employee UI end-to-end.
M9	Security, observability and production hardening.
M10	GitHub Actions + Docker + Cloud Run deployment.
M11	Final QA and MVP release.

Definition of Done

- Implementation complete; Zod and authorization checks included where applicable.
- Tests pass, including relevant regression tests.
- Sensitive mutations are audited.
- Documentation updated and CI passes.
- Known defects are fixed or documented with an owner.

MVP acceptance criteria

- Administrators manage organizations, employees, compensation and payroll inputs.
- Payroll calculates versioned statutory/configured rules and supports approval/finalization.
- Payslips are stored in GCP Cloud Storage and securely accessible.
- Reports use finalized payroll data; scheduled work runs through BullMQ/Redis.
- Email works through Nodemailer/SendGrid; deployment runs through GitHub Actions and Cloud Run.

Decisions to lock

Select one PDF engine for MVP. Pin dependency versions. Validate the Cloud Run worker execution model. Recheck official Kenya payroll rules before production. The system must be repeatable, auditable, testable, explainable and safe to change.